

平成 2 7 年度

筑波大学大学院博士課程
システム情報工学研究科
コンピュータサイエンス専攻

博士前期課程（一般入学試験 8 月期）

試験問題 **基礎科目（情報基礎）**

Fundamentals of Computer Science

[注意事項][Instructions]

1. 試験開始の合図があるまで、問題の中を見てはいけません。また、筆記用具を手に持っていないけません。
Do NOT open this booklet before the examination starts. In addition, do not have a pen in hand.
2. 試験開始の合図のあとで、全ての解答用紙の定められた欄に、研究科、専攻、受験番号を記入すること。
Fill in the designated spaces on each answer sheet with the name of the graduate school, name of your main (major) field, and the examination number after the examination starts.
3. この問題は全部で 10 ページ（表紙を除く）です。1～5 ページは日本語版、6～10 ページは英語版です。
This booklet consists of 10 pages, excluding the cover sheet. The Japanese version is shown on pages 1-5 and the English version on pages 6-10.
4. 解答用紙（罫線有り）を 2 枚、下書き用紙（白紙）を 1 枚配布します。
You are given two ruled answer sheets and one white draft sheet.
5. 問題は全部で 2 問あります。問題 I の解答を 1 枚目、問題 II の解答を 2 枚目の解答用紙に、必ず分けて記入すること。
There are two problems. Write the answers to Problem I on the first answer sheet and the answers to Problem II on the second answer sheet.
6. 解答用紙に解答を記述する際に、問題 I の解答か問題 II の解答かを必ず明記すること。
When writing the answers, clearly label the problem number on each answer sheet.

平成 2 6 年 8 月 2 1 日

問題 I の解答は 1 枚目の解答用紙に記入すること。問題 I の解答であることを明記すること。

問題 I ノードが整数値を持つ 2 分木をポインタを用いて実現する。図 1 は 2 分木を表すデータ構造を箱と矢印を用いて図示したものである。

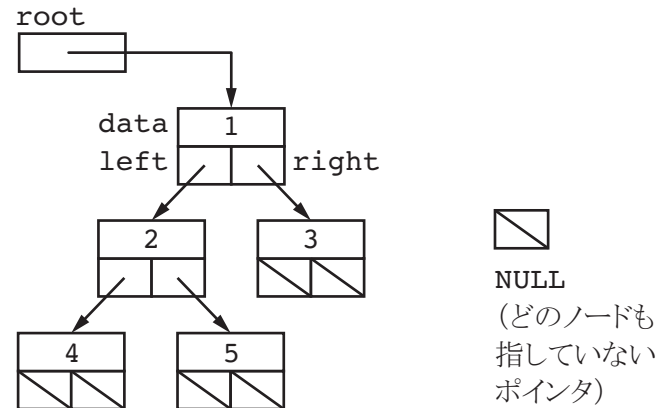


図 1: 2 分木を表現したデータ構造を図示した例

2 分木を図 2 に示したような C 言語の構造体と大域変数を用いて記述する。変数 `root` は 2 分木の根を保持しており、初期値は `NULL` である。定数 `NULL` は、どのノードも指していないポインタを意味する。図 3 はいくつかの関連する関数の定義を示している。なお、関数 `malloc()` は指定されたバイト数のメモリを割り当て、そのポインタを返す関数である。ここではメモリの割り当ては常に成功するものとする。

```
struct node {
    int data;
    struct node *left;
    struct node *right;
}

struct node *root = NULL;
```

図 2: 2 分木を表す C 言語の構造体と大域変数

以下の設問に答えなさい。

- (1) 関数 `make_sample_tree` によって作られる木を、図 1 の表記に従い図示しなさい。
- (2) 図 4 に図示したデータ 1 に対して、関数が `sumup_list1(root, 0)` のように呼び出された時、実行後のデータを図 1 の表記に従い図示しなさい。
- (3) 図 5 に図示したデータ 2 に対して、関数が `sumup_list2(root)` のように呼び出された時、実行後のデータを図 1 の表記に従い図示しなさい。

- (4) 深さ n の完全 2 分木に対して、関数が `sumup_tree1(root, 0)` のように呼び出された時、(A) の行の代入が根 (`root` の指すノード) に対して行われるまでに、最初の呼び出しも含めて関数 `sumup_tree1` は何回呼び出されるか答えなさい。なお、「完全 2 分木」とは、すべての内部ノード（葉でないノード）が 2 つの子を持ち、すべての葉が同じ深さを持つ 2 分木とする。根だけからなる 2 分木の深さは 0 である。
- (5) 図 2 で定義した 2 分木のデータ構造を利用して、根つき木を表すことにする。図 6 はその表し方を示している。ポインタ `left` が 1 番目の子を指し、ポインタ `right` が次の兄弟を指す。関数を `sumup_tree2(root)` のように呼び出すと、根つき木をボトムアップに探索して、子の値の合計を親の値に加える。図 7 は値が更新される様子を示している。図 3 の (B) と (C) の部分を埋めて、関数 `sumup_tree2` を完成させなさい。

```

struct node *make_node(int value, struct node *left, struct node *right) {
    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->data = value;
    p->left = left;
    p->right = right;
    return p;
}

void make_sample_tree(void) {
    struct node *n2, *n3;

    n3 = make_node(3, NULL, NULL);
    n2 = make_node(2, NULL, n3);
    root = make_node(1, n2, NULL);
}

int sumup_list1(struct node *p, int value) {
    while (p != NULL) {
        p->data += value;
        value = p->data;
        p = p->right;
    }
    return value;
}

int sumup_list2(struct node *p) {
    if (p == NULL) return 0;
    p->data += sumup_list2(p->right);
    return p->data;
}

int sumup_tree1(struct node *p, int value) {
    if (p == NULL) return value;
    p->data += sumup_tree1(p->left, value);    /* (A) */
    return sumup_tree1(p->right, p->data);
}

int sumup_tree2(struct node *p) {
    (B)
    p->data += sumup_tree2(p->left);
    (C)
}

```

図 3: 関数の定義

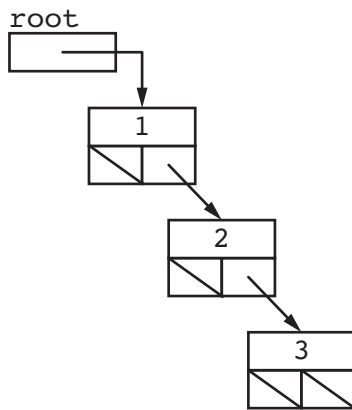


図 4: データ 1

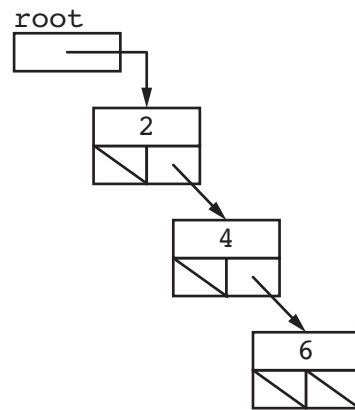
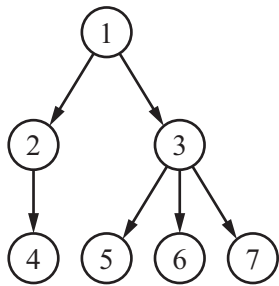
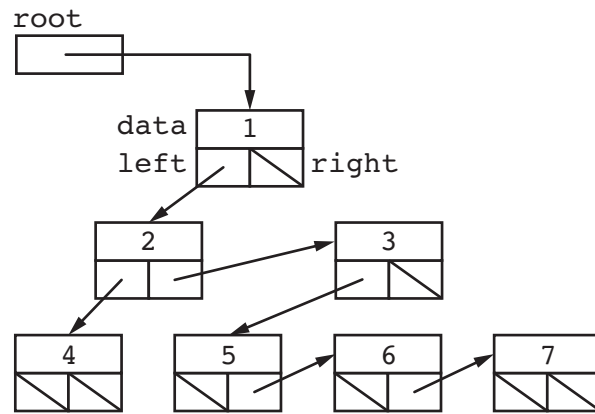


図 5: データ 2



(a) ノードが整数値を持つ根つき木の例



(b) 根つき木を表現するデータ構造を図示した例

図 6: 2分木による根つき木の表現方法

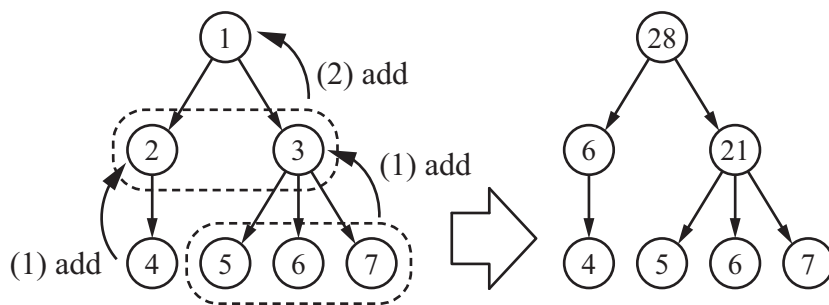


図 7: 根つき木の処理例

問題 II の解答は 2 枚目の解答用紙に記入すること。問題 II の解答であることを明記すること。

問題 II デジタル回路に関して考える。下の表に示すビットパターンが与えられた場合、以下の設問に答えよ。

a	b	c	d	e	f	g
0	1	0	1	1	1	0
1	0	1	0	1	0	1
0	1	0	0	0	1	0
0	0	1	1	0	1	1
1	0	1	1	1	1	0
1	0	1	1	0	1	0
0	1	0	0	1	0	0
1	1	0	0	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	1	0

(1) a と b を入力とし、c を出力とする論理式を示せ。

(2) d と e を入力とし、f を出力とする論理式を示せ。

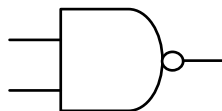
(3) a、b、c、d を入力とし、g を出力とする場合を考える。

(3-1) g に関する下記カルノー図を埋めて完成させよ。上の表にない入出力パターンについては「*」を記入せよ。

ab \ cd	00	01	11	10
00				
01	0			
11				
10		*		1

(3-2) g に関して、入力信号が 3 つになるようにし、項数が最少になる論理式を 1 つ示せ。「*」のところの出力は 0 でも 1 でもよい。

(3-3) (3-2) で求めた論理式をできるだけ少ない個数の 2 入力 1 出力の NAND ゲートのみを使って実現する回路図をかけ。ただし、NAND ゲートの図は下記を用いるものとする。



Write the answers to Problem I on the first answer sheet, and clearly label it at the top of the page as “Problem I.”

Problem I We realize a binary tree whose nodes have integers by using pointers. Figure 1 shows an example illustration of the binary tree in a boxes and arrows notation.

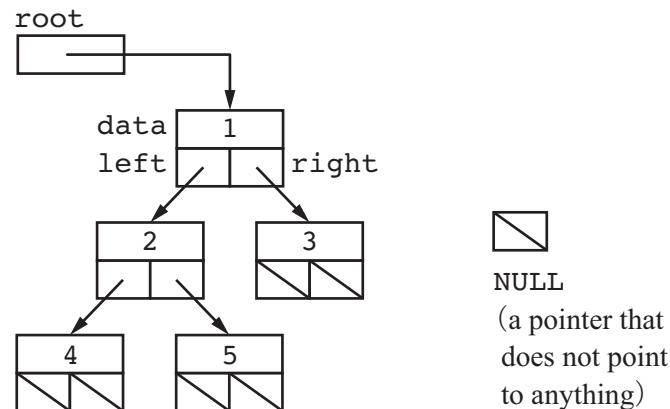


Figure 1: An example illustration of the data represented by a binary tree

We describe a binary tree by using the structure and the global variable in the C language as shown in Figure 2. The variable `root` holds the root of the binary tree, and it is initialized with `NULL`. The constant `NULL` means a pointer that does not point to anything. Figure 3 shows definitions of some functions. Note that `malloc()` is a function that allocates a memory block of the specified size (in bytes) and returns its pointer. It is assumed that the memory allocation always succeeds.

```

struct node {
    int data;
    struct node *left;
    struct node *right;
}

struct node *root = NULL;

```

Figure 2: The structure and the global variable that implement a binary tree in the C language

Answer the following questions.

- (1) Illustrate the binary tree created by function `make_sample_tree` in a boxes and arrows notation shown in Figure 1.
- (2) When function `sumup_list1(root, 0)` is applied to Data 1 shown in Figure 4, illustrate the data after calling the function in a boxes and arrows notation shown in Figure 1.
- (3) When function `sumup_list2(root)` is applied to Data 2 shown in Figure 5, illustrate the data after calling the function in a boxes and arrows notation shown in Figure 1.

- (4) When function `sumup_tree1(root, 0)` is applied to the perfect binary tree with depth n , how many times is the function `sumup_tree1` called, including the first call, until the assignment in line (A) for the root (the node pointed to by `root`) is executed? A *perfect binary tree* is a binary tree in which every internal node (node that is not a leaf) has two children and all leaves have the same depth. The depth of a binary tree composed of only a root is 0.
- (5) Let us represent a rooted tree by using the structure for a binary tree defined in Figure 2. Figure 6 illustrates the way to represent a rooted tree. The pointer `left` points to the first child and the pointer `right` points to the next sibling. Function call `sumup_tree2(root)` traverses the rooted tree bottom-up, and adds childrens' data to their parent data as illustrated in Figure 7. Fill in the boxes (B) and (C) in Figure 3 to complete the function `sumup_tree2`.

```

struct node *make_node(int value, struct node *left, struct node *right) {
    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->data = value;
    p->left = left;
    p->right = right;
    return p;
}

void make_sample_tree(void) {
    struct node *n2, *n3;

    n3 = make_node(3, NULL, NULL);
    n2 = make_node(2, NULL, n3);
    root = make_node(1, n2, NULL);
}

int sumup_list1(struct node *p, int value) {
    while (p != NULL) {
        p->data += value;
        value = p->data;
        p = p->right;
    }
    return value;
}

int sumup_list2(struct node *p) {
    if (p == NULL) return 0;
    p->data += sumup_list2(p->right);
    return p->data;
}

int sumup_tree1(struct node *p, int value) {
    if (p == NULL) return value;
    p->data += sumup_tree1(p->left, value);    /* (A) */
    return sumup_tree1(p->right, p->data);
}

int sumup_tree2(struct node *p) {
    (B)
    p->data += sumup_tree2(p->left);
    (C)
}

```

Figure 3: Definitions of functions

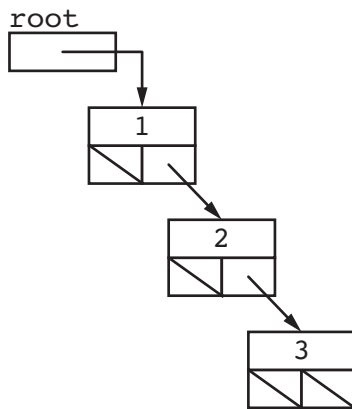


Figure 4: Data 1

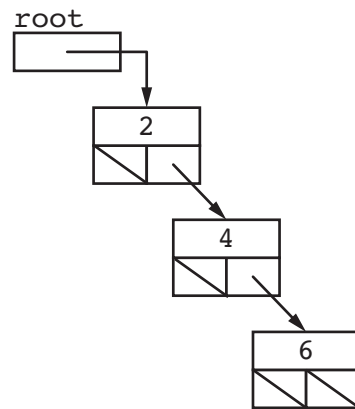


Figure 5: Data 2

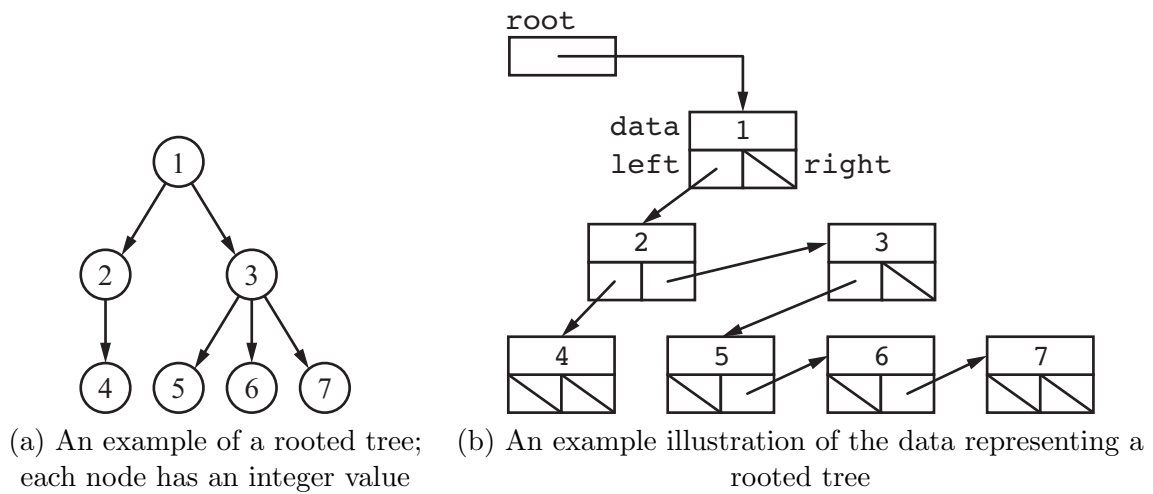


Figure 6: Representation of a rooted tree by using a binary tree

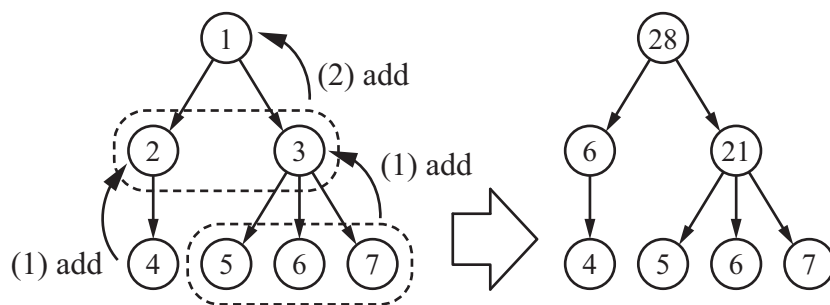


Figure 7: An example of processing a rooted tree

Write the answers to Problem II on the second answer sheet, and clearly label it at the top of the page as “Problem II.”

Problem II Let us consider a digital circuit. For the bit pattern given in the table below, answer the following questions.

a	b	c	d	e	f	g
0	1	0	1	1	1	0
1	0	1	0	1	0	1
0	1	0	0	0	1	0
0	0	1	1	0	1	1
1	0	1	1	1	1	0
1	0	1	1	0	1	0
0	1	0	0	1	0	0
1	1	0	0	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	1	0

- (1) When a and b are input signals and c is the output, derive a logical formula for c in terms of a and b.
- (2) When d and e are input signals and f is the output, derive a logical formula for f in terms of d and e.
- (3) Let a, b, c and d be input signals and g the output one.
 - (3-1) Complete the Karnaugh map below for g. If input/output pattern(s) are not included in the above table, write “*”.

ab \ cd	00	01	11	10
00				
01	0			
11				
10		*		1

- (3-2) Regarding g, write a logical statement that includes the minimum number of terms. Here the number of inputs must be three. The output pattern(s) marked by “*” can be regarded as 0 or 1.
- (3-3) Derive a schematic for the logical statement in your answer to question (3-2). Use only NAND gates as few as possible. Here, use the symbol of the NAND gate given below.

